

DATABASE MANAGEMENT SYSTEM PROJECT REPORT

Submitted by:

Ayush Jindal 1024030748

Tanish Bansal 1024030742

Aditya Kumar Srivastava 1024030745

BE Second Year, COE

Submitted to: Dr. BANISHA SHARMA



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

TABLE OF CONTENTS

SR.NO.	CONTENTS
1	INTRODUCTION
2	LITERATURE REVIEW
3	METHODOLOGY
4	IMPLEMENTATION
5	RESULT
6	CONCLUSION
7	REFERENCES

Introduction

Background

Students often face difficulty finding available seats in libraries, especially during exams. This leads to time wastage and inefficient seat utilization.

Objective

- Provide real-time seat availability
- Automate seat allocation using ID scanning
- Reduce manual searching effort
- Maintain accurate occupancy records

Scope

- Works within a multi-floor library
- Supports real-time allocation and release
- Can be extended to mobile/web applications

Limitations

- Depends on correct scanning behaviour
- No reservation system implemented

LITERATURE REVIEW

EXISTING SYSTEM

- Currently our campus library has scanner entry and exit but it doesn't maintain seat tracking and allocation and this where our project comes in terms of enhancing the already existing system

COMPARISON

FEATURES	Existing System	Proposed System
Scanner	Exists	Needs to be integrated with already existing system.
Seat Tracking	No	Yes
Real-Time Updates	No	Yes

RESEARCH GAP

- The current System lags simple tracking of seat allocation, which can help students a lot during peak exams time to track no of seats left to choose and they be able to plan their visits to library and by also maintaining a time stamp of how long seats are reserved by student authority can track total usage of library resources.

METHODOLOGY

Approach

- Students scan ID at entry
- System logs scan → trigger activates
- Procedure allocates seat
- Exit scan releases seat

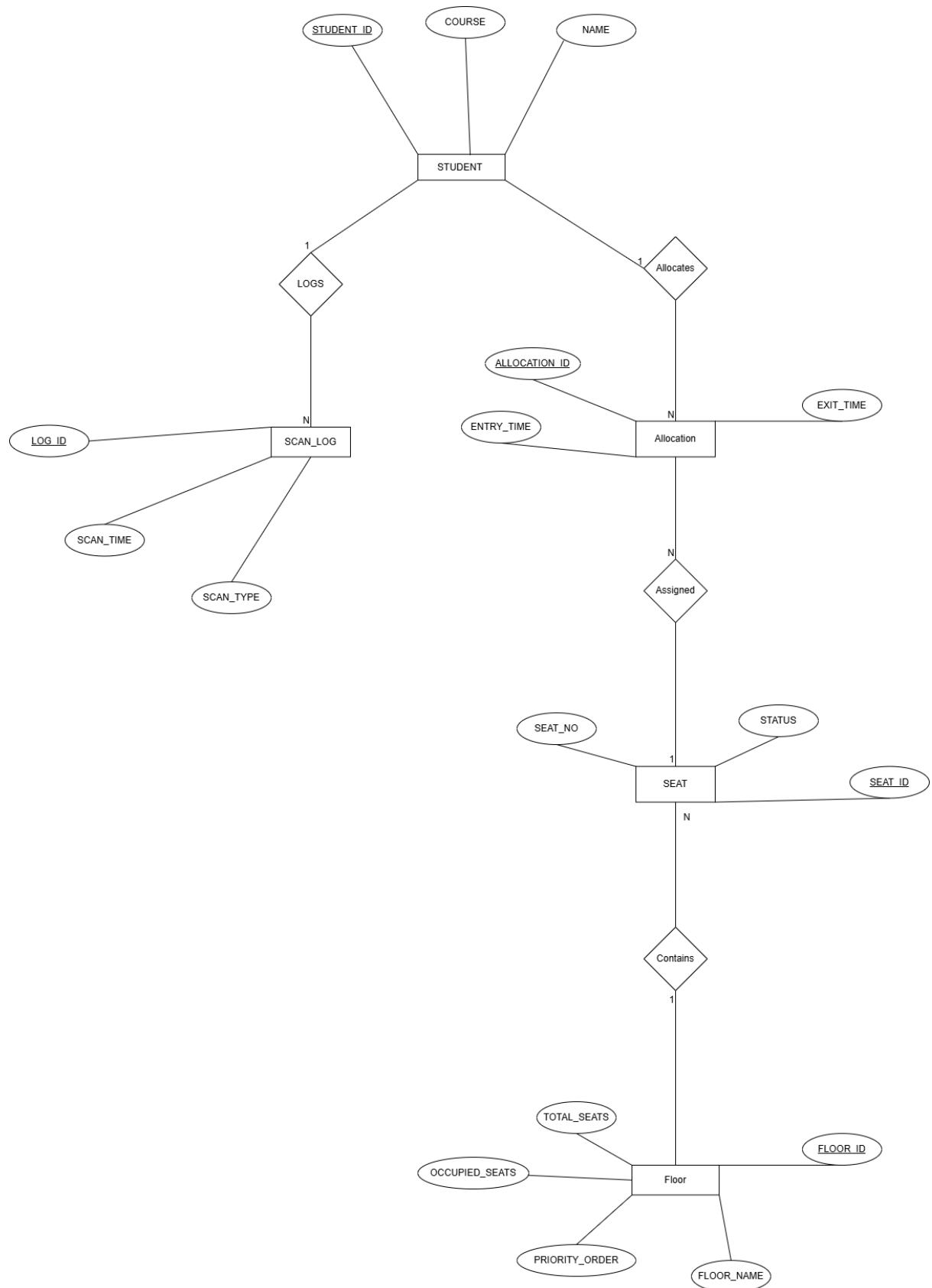
ER RELATIONSHIP

ENTITY 1	Name of the relationship	ENTITY 2	Cardinality
Student	Allocates	Allocation	1:N
Student	Logs	Scan_Log	1:N
Allocation	Assigned	Seats	N:1
Seat	Contains	Floor	N:1

DBMS Concepts Used

- **Triggers** → automatic execution
- **Procedures** → business logic
- **Primary Keys** → uniqueness
- **Foreign Keys** → relationships
- **Normalization** → avoids redundancy
- **ACID properties** → data consistency

ER-DIAGRAM



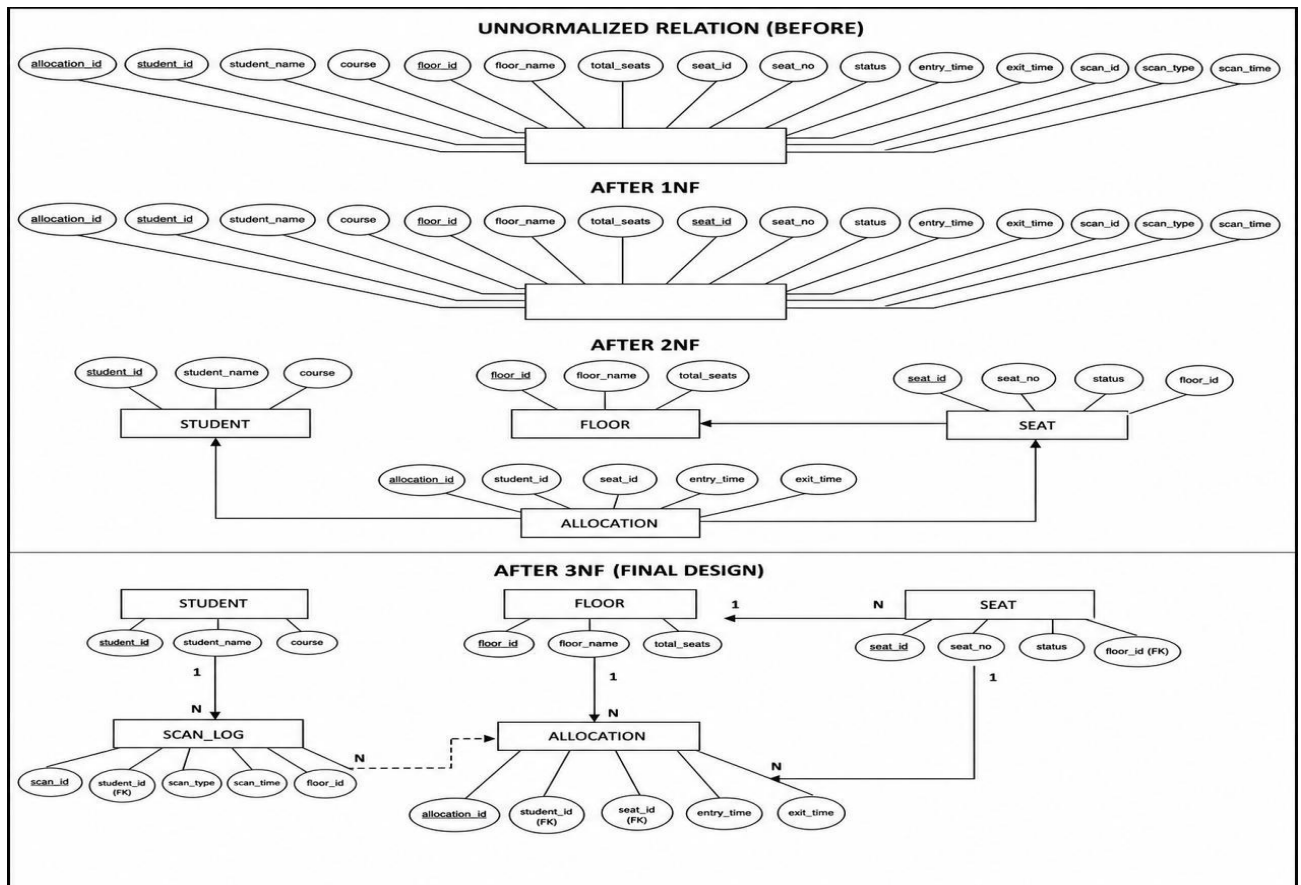
IMPLEMENTATION

NORMALIZATION

The database schema has been normalized up to Third Normal Form (3NF). Each table ensures atomicity, eliminates partial dependencies, and avoids transitive dependencies, thereby maintaining data integrity and reducing redundancy.

SR.NO	Functional dependencies	Normalization
1	$\text{student_id} \rightarrow \text{student_name, course}$	3NF
2	$\text{floor_id} \rightarrow \text{floor_name, total_seats, occupied_seats, priority_order, status}$	3NF
3	$\text{seat_id} \rightarrow \text{floor_id, seat_no, status}$	3NF
4	$\text{allocation_id} \rightarrow \text{student_id, seat_id, entry_time, exit_time}$	3NF
5	$\text{scan_id} \rightarrow \text{student_id, scan_type, scan_time, floor_id}$	3NF

Relational Diagram



Databases

1.Student Table

```
CREATE TABLE student (  
    student_id VARCHAR2(20) PRIMARY KEY,  
    student_name VARCHAR2(50),  
    course VARCHAR2(30)  
);
```

	STUDENT_ID	STUDENT_NAME	COURSE
1	STU101	Ayush	CSE
2	STU102	Rahul	CSE
3	STU107	New Student	CSE

2.Allocation Table

```
CREATE TABLE allocation (  
    allocation_id NUMBER PRIMARY KEY,  
    student_id VARCHAR2(20),  
    seat_id NUMBER,  
    entry_time TIMESTAMP,  
    exit_time TIMESTAMP,  
    FOREIGN KEY (student_id) REFERENCES student(student_id),  
    FOREIGN KEY (seat_id) REFERENCES seat(seat_id)  
);
```

	ALLOCATION_ID	STUDENT_ID	SEAT_ID	ENTRY_TIME	EXIT_TIME
1	21	STU101		1 2026-04-29T10:27: (null)	
2	1	STU101		1 2026-04-29T10:26: 2026-04-29T10:26:	
3	2	STU102		2 2026-04-29T10:28: (null)	
4	22	STU107		201 2026-04-29T10:36: (null)	

3.Seat Table

```
CREATE TABLE seat (
    seat_id NUMBER PRIMARY KEY,
    floor_id NUMBER,
    seat_no NUMBER,
    status VARCHAR2(10),
    FOREIGN KEY (floor_id) REFERENCES floor(floor_id)
);
```

	SEAT_ID	FLOOR_ID	SEAT_NO	STATUS
1	716	4	116	FREE
2	717	4	117	FREE
3	718	4	118	FREE
4	719	4	119	FREE
5	720	4	120	FREE
6	721	4	121	FREE

4.Floor Table

```
CREATE TABLE floor (
    floor_id NUMBER PRIMARY KEY,
    floor_name VARCHAR2(20),
    total_seats NUMBER,
    occupied_seats NUMBER DEFAULT 0,
    priority_order NUMBER,
    status VARCHAR2(10)
);
```

Download Execution time: 0.01 seconds

	FLOOR_NAME	TOTAL_SEATS	OCCUPIED_SEATS	PRIORITY_ORDER	STATUS
1	1 Ground	200	2	1	OPEN
2	2 First	200	1	2	OPEN
3	3 Second	200	0	3	OPEN
4	4 Third	200	0	4	OPEN
5	5 Fourth	200	0	5	OPEN

5.Scan_Log Table

```
CREATE TABLE scan_log (
    scan_id NUMBER PRIMARY KEY,
    student_id VARCHAR2(20),
    scan_type VARCHAR2(10),
    scan_time TIMESTAMP
);
```

Download Execution time: 0.013 seconds

	SCAN_ID	STUDENT_ID	SCAN_TYPE	SCAN_TIME	FLOOR_ID
1	21	STU101	ENTRY	2026-04-29T10:27:51	(null)
2	2	STU101	EXIT	2026-04-29T10:28:00	(null)
3	1	STU101	ENTRY	2026-04-29T10:28:01	(null)
4	3	STU999	EXIT	2026-04-29T10:28:02	(null)
5	5	STU102	ENTRY	2026-04-29T10:28:03	(null)
6	23	STU107	ENTRY	2026-04-29T10:36:00	2

PROCEDURES

1. Allocate Seat

```
CREATE OR REPLACE PROCEDURE allocate_seat_floor(
    p_student_id IN VARCHAR2,
    p_floor_id IN NUMBER
)
AS
    v_seat_id NUMBER;
    v_count NUMBER;
BEGIN
    -- Check student exists
    SELECT COUNT(*) INTO v_count
    FROM student
    WHERE student_id = p_student_id;
```

```

IF v_count = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Student does not exist');
    RETURN;
END IF;

-- Check if already inside
SELECT COUNT(*) INTO v_count
FROM allocation
WHERE student_id = p_student_id
AND exit_time IS NULL;

IF v_count > 0 THEN
    DBMS_OUTPUT.PUT_LINE('Student already inside');
    RETURN;
END IF;

-- Get free seat
SELECT seat_id INTO v_seat_id
FROM seat
WHERE floor_id = p_floor_id
AND status = 'FREE'
ORDER BY seat_no
FETCH FIRST 1 ROWS ONLY;

-- Insert allocation
INSERT INTO allocation
VALUES(alloc_seq.NEXTVAL, p_student_id, v_seat_id, SYSTIMESTAMP, NULL);

-- Update seat
UPDATE seat SET status = 'OCCUPIED'
WHERE seat_id = v_seat_id;

-- Update floor
UPDATE floor
SET occupied_seats = occupied_seats + 1
WHERE floor_id = p_floor_id;

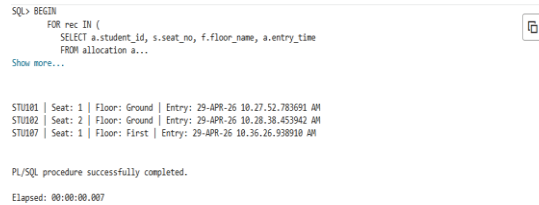
DBMS_OUTPUT.PUT_LINE('Seat allocated to ' || p_student_id ||
    ' | Seat ID: ' || v_seat_id ||
    ' | Floor: ' || p_floor_id);

```

```

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No seats available on selected floor');
END;
/

```



SQL> BEGIN
FOR rec IN (
SELECT a.student_id, s.seat_no, f.floor_name, a.entry_time
FROM allocation a...

Show more...

STU101	Seat: 1	Floor: Ground	Entry: 29-APR-26 10.27.52.783691 AM
STU102	Seat: 2	Floor: Ground	Entry: 29-APR-26 10.28.38.453942 AM
STU107	Seat: 1	Floor: First	Entry: 29-APR-26 10.36.26.938910 AM

PL/SQL procedure successfully completed.
Elapsed: 00:00:00.007

2.Release Seat

```

CREATE OR REPLACE PROCEDURE release_seat(
    p_student_id IN VARCHAR2
)

```

AS

```

    v_seat_id NUMBER;

```

```

    v_floor_id NUMBER;

```

BEGIN

```

    -- Get active allocation

```

```

    SELECT seat_id INTO v_seat_id

```

```

    FROM allocation

```

```

    WHERE student_id = p_student_id

```

```

    AND exit_time IS NULL;

```

```

    -- Mark exit

```

```

    UPDATE allocation

```

```

    SET exit_time = SYSTIMESTAMP

```

```

    WHERE student_id = p_student_id

```

```

    AND exit_time IS NULL;

```

```

-- Get floor

SELECT floor_id INTO v_floor_id
FROM seat
WHERE seat_id = v_seat_id;

-- Free seat

UPDATE seat SET status = 'FREE'
WHERE seat_id = v_seat_id;

-- Update floor

UPDATE floor
SET occupied_seats = occupied_seats - 1
WHERE floor_id = v_floor_id;

DBMS_OUTPUT.PUT_LINE('Seat released for ' || p_student_id ||
    ' | Seat ID: ' || v_seat_id ||
    ' | Floor: ' || v_floor_id);

EXCEPTION

WHEN NO_DATA_FOUND THEN

    DBMS_OUTPUT.PUT_LINE('No active allocation found for ' || p_student_id);

END;

/

```

```
release_seat('STU001');
release_seat('STU002');
release_seat('STU003');...
```

Show more...

No active allocation found for STU001
Seat released for STU002 | Seat ID: 2 | Floor: 1
No active allocation found for STU003

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.006

TRIGGERS

1. SCAN_TRIGGER

CREATE OR REPLACE TRIGGER scan_trigger

AFTER INSERT ON scan_log

FOR EACH ROW

BEGIN

IF :NEW.scan_type = 'ENTRY' THEN

allocate_seat_floor(:NEW.student_id, :NEW.floor_id);

ELSIF :NEW.scan_type = 'EXIT' THEN

release_seat(:NEW.student_id);

END IF;

END;

/

Download ▾ Execution time: 0.095 seconds			
	TRIGGER_NAME	TABLE_NAME	STATUS
1	SCAN_TRIGGER	SCAN_LOG	ENABLED

(Trigger Properties)

	ALLOCATION_ID	STUDENT_ID	SEAT_ID	ENTRY_TIME	EXIT_TIME
1	43	STU101	202	2026-05-01T06:36:24	(null)

(Entry time updated by trigger)

	ALLOCATION_ID	STUDENT_ID	SEAT_ID	ENTRY_TIME	EXIT_TIME
1	43	STU101	202	2026-05-01T06:36:24	2026-05-01T06:37:51
2	42	STU101	202	2026-05-01T06:36:24	2026-05-01T06:36:24
3	41	STU101	202	2026-05-01T06:34:24	2026-05-01T06:34:24
4	21	STU101	1	2026-04-29T10:27:51	2026-05-01T06:28:24
5	1	STU101	1	2026-04-29T10:26:24	2026-04-29T10:26:24

(Exit Time updated by Trigger)

CURSORS

Note: Are project doesn't use any explicit cursors but there are implicit cursors used which are mentioned below

1. Check Duplicate Entry-

```
SELECT COUNT(*) INTO v_count
FROM allocation
WHERE student_id = p_student_id
AND exit_time IS NULL;
```

2. Find Free Seat-

```
SELECT seat_id INTO v_seat_id
FROM seat
WHERE floor_id = p_floor_id
AND status = 'FREE'
ORDER BY seat_no
FETCH FIRST 1 ROWS ONLY;
```

3. Update Seat Status-

```
UPDATE seat
SET status = 'OCCUPIED'
WHERE seat_id = v_seat_id;
```

4. Update Floor Count-

```
UPDATE floor
SET occupied_seats = occupied_seats + 1
WHERE floor_id = p_floor_id;
```

5. Get Seat from Allocation-

```
SELECT seat_id INTO v_seat_id
FROM allocation
WHERE student_id = p_student_id
AND exit_time IS NULL;
```

6. Update Exit Time-

```
UPDATE allocation
```

```
SET exit_time = SYSTIMESTAMP  
WHERE student_id = p_student_id  
AND exit_time IS NULL;
```

7. Get Floor Id-

```
SELECT floor_id INTO v_floor_id  
FROM seat  
WHERE seat_id = v_seat_id;
```

8. Free Seat-

```
UPDATE seat  
SET status = 'FREE'  
WHERE seat_id = v_seat_id;
```

9. Update Floor Count-

```
UPDATE floor  
SET occupied_seats = occupied_seats - 1  
WHERE floor_id = v_floor_id;
```

RESULTS

1. Insert data into Scan_Log table which will be basically done

In real world by scanning our admit card

```
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU201','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU202','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU203','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU204','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU205','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU206','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU207','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU208','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU209','ENTRY',SYSTIMESTAMP,2);
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU210','ENTRY',SYSTIMESTAMP,2);
```

COMMIT;

Seat allocated to STU201 Seat ID: 202 Floor: 2	 
Seat allocated to STU202 Seat ID: 2 Floor: 1	 
Seat allocated to STU203 Seat ID: 203 Floor: 2	
Seat allocated to STU204 Seat ID: 402 Floor: 3	 
Seat allocated to STU205 Seat ID: 602 Floor: 4	 
Seat allocated to STU206 Seat ID: 802 Floor: 5	
Seat allocated to STU207 Seat ID: 403 Floor: 3	 
Seat allocated to STU208 Seat ID: 803 Floor: 5	 
Seat allocated to STU209 Seat ID: 3 Floor: 1	 
Seat allocated to STU210 Seat ID: 603 Floor: 4	 

2. Number of Active Allocations

```
SELECT student_id, seat_id, entry_time
FROM allocation
WHERE exit_time IS NULL
ORDER BY student_id;
```

	STUDENT_ID	SEAT_ID	ENTRY_TIME
1	STU201	202	2026-05-01T09:05:23.000000000
2	STU202	2	2026-05-01T09:05:23.000000000
3	STU203	203	2026-05-01T09:05:23.000000000
4	STU204	402	2026-05-01T09:05:23.000000000
5	STU205	602	2026-05-01T09:05:23.000000000
6	STU206	802	2026-05-01T09:05:23.000000000
7	STU207	403	2026-05-01T09:05:23.000000000
8	STU208	803	2026-05-01T09:05:23.000000000
9	STU209	3	2026-05-01T09:05:23.000000000
10	STU210	603	2026-05-01T09:05:23.000000000

3. Floor Occupancy

```
SELECT floor_id, occupied_seats
FROM floor;
```

	FLOOR_ID	OCCUPIED_SEATS
1	1	2
2	2	2
3	3	2
4	4	2
5	5	2

4. Duplicate entry (validity check)

```
INSERT INTO scan_log
VALUES(scan_seq.NEXTVAL,'STU201','ENTRY',SYSTIMESTAMP,2);

COMMIT;
```

```
SQL> INSERT INTO scan_log
VALUES(scan_seq.NEXTVAL,'STU201','ENTRY',SYSTIMESTAMP,2)
```



```
Student already inside
```

```
1 row inserted.
```

```
Elapsed: 00:00:00.007
```

5. EXIT

```
INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU201','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU202','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU203','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU204','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU205','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU206','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU207','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU208','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU209','EXIT',SYSTIMESTAMP,2);

INSERT INTO scan_log VALUES(scan_seq.NEXTVAL,'STU210','EXIT',SYSTIMESTAMP,2);

COMMIT;
```

Seat released for STU201 Seat ID: 202 Floor: 2	
Seat released for STU202 Seat ID: 2 Floor: 1	
Seat released for STU203 Seat ID: 203 Floor: 2	
Seat released for STU204 Seat ID: 402 Floor: 3	
Seat released for STU205 Seat ID: 602 Floor: 4	
Seat released for STU206 Seat ID: 802 Floor: 5	
Seat released for STU207 Seat ID: 403 Floor: 3	
Seat released for STU208 Seat ID: 803 Floor: 5	
Seat released for STU209 Seat ID: 3 Floor: 1	
Seat released for STU210 Seat ID: 603 Floor: 4	

CONCLUSION

The Library Seat Allocation Management System developed in this project demonstrates an effective application of database management concepts to solve a real-world problem. The system successfully automates the process of seat allocation and release in a multi-floor library environment, reducing manual effort and minimizing time wastage for students.

By integrating core DBMS features such as tables, primary and foreign key constraints, triggers, and stored procedures, the system ensures data consistency, integrity, and real-time updates. The use of triggers allows automatic execution of allocation and release logic whenever a scan event is recorded, thereby eliminating the need for manual intervention. Additionally, the implementation of business rules—such as preventing duplicate seat allocation for a single student—ensures reliability and correctness of the system.

The project also highlights the importance of referential integrity and normalization, which help maintain a structured and efficient database. The system is capable of tracking seat occupancy, managing floor-wise availability, and maintaining a history of student usage through allocation records. This makes it not only functional but also analytically useful.

From a practical perspective, the system significantly improves user experience by providing a structured mechanism for seat management. Students no longer need to search for available seats manually, and administrators can monitor occupancy levels easily.

In conclusion, the project successfully achieves its objective of designing a robust, database-driven solution for library seat management while demonstrating key DBMS concepts in a practical and meaningful way.

REFERENCES

- 1.Queries Execution-<https://freesql.com/>
- 2.ER DIAGRAM MAKING -<https://app.diagrams.net/>
3. Relational Diagram-<https://app.diagrams.net/>
- 3.Lecture Notes